

SALESTORM

SYSCRAFTERS 2026 · Design-First AI-Assisted System Design Hackathon

Complete System Design Documentation

10,000 concurrent users · 100 units · Zero overselling

1. Architecture Summary & Key Numbers

SALESTORM is a flash-sale platform that admits at most 100 successful reservations under 10,000 concurrent buy attempts without overselling. Payment and order creation remain consistent under partial outages via transactional outbox and idempotent consumers.

Key Numbers

Metric	Target / Design
Concurrent users	10,000
Inventory units	100
Overselling	0 (Redis Lua + MySQL UPDATE + CHECK)
Reservation TTL	10 minutes
p99 reserve latency	< 200 ms
Out-of-stock reject	~0.1 ms Redis only (zero MySQL)
Payment → Order	Transactional Outbox + Debezium + Kafka
Idempotency	X-Idempotency-Key + UNIQUE(reservation_id)

Architecture Highlights

- Redis Lua atomic stock gate — single-threaded, race-free admission
- MySQL conditional UPDATE + CHECK — durable source of truth
- End-to-end idempotency — no double charge or double order
- Transactional Outbox + Kafka — payment success always becomes an order
- Fail-closed on Redis outage — sale pauses, never oversells
- Multi-AZ Kubernetes with HPA — scale gateway/inventory under load

2. System Context

Customer interacts with SALESTORM for browse, reserve, pay and track. External systems: Payment Gateway (Razorpay/Stripe), Notification Provider, Shipment Partner. Card data never enters the platform (PCI scope minimized). Notifications and shipment are async and never block the critical path.

System Context - SALESTORM Flash Sale

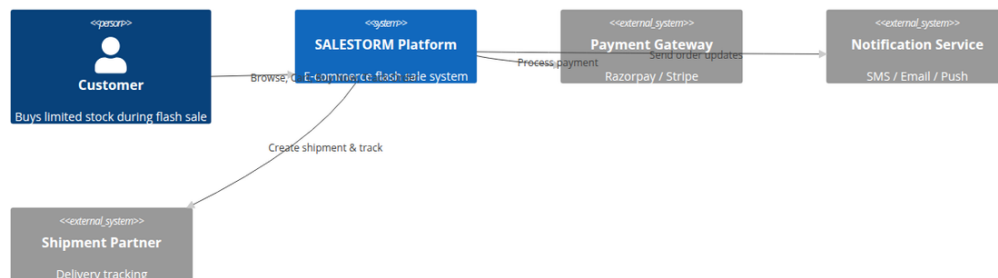


Figure 1 — System Context

3. High-Level Architecture

Traffic path: Client → CDN/WAF → Edge Rate Limiter → API Gateway → JWT Auth → Product / Inventory / Payment services. Inventory uses Redis for hot stock counters and MySQL for durable truth. Payment writes payment + outbox in one DB transaction; Debezium streams to Kafka; Order service consumes idempotently and creates the order.

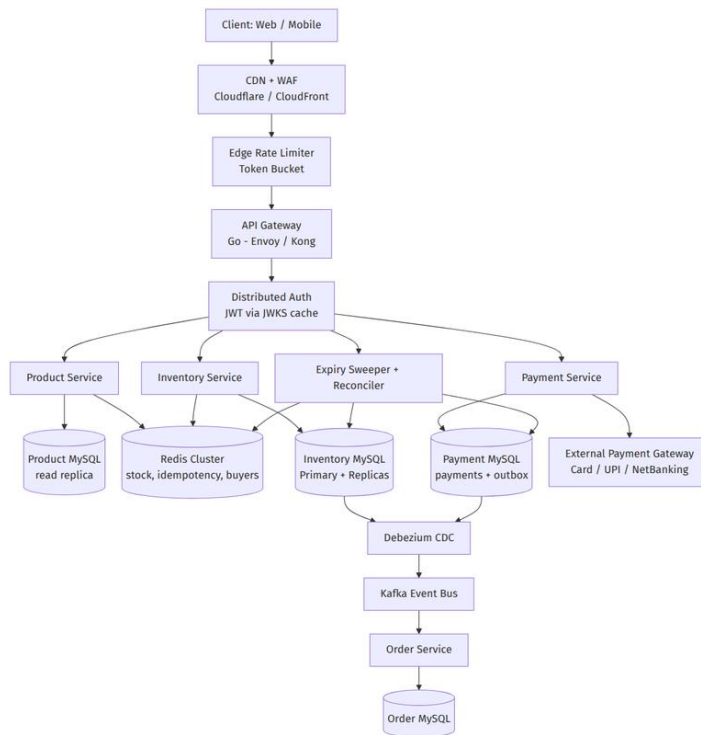


Figure 2 — HLD Architecture

3.1 Container Diagram

Platform containers (Gateway, Inventory, Payment, Order, Notification, Sweeper), data stores (Redis, three MySQL DBs), and async pipeline (Debezium + Kafka) with external systems.

Container Diagram — SALESTORM Platform

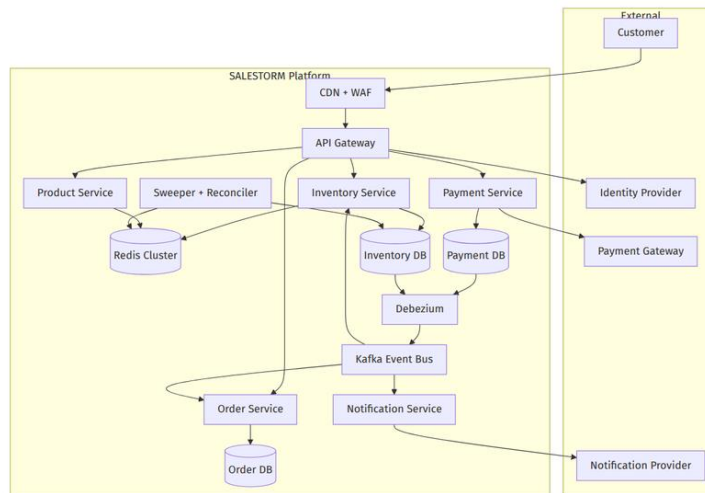


Figure 3 — Container Diagram

3.2 Deployment Multi-AZ

Region ap-south-1 (Mumbai), three Availability Zones. HPA on CPU and RPS. MySQL primary with semi-sync replicas. Redis primary + replica. Kafka three brokers.

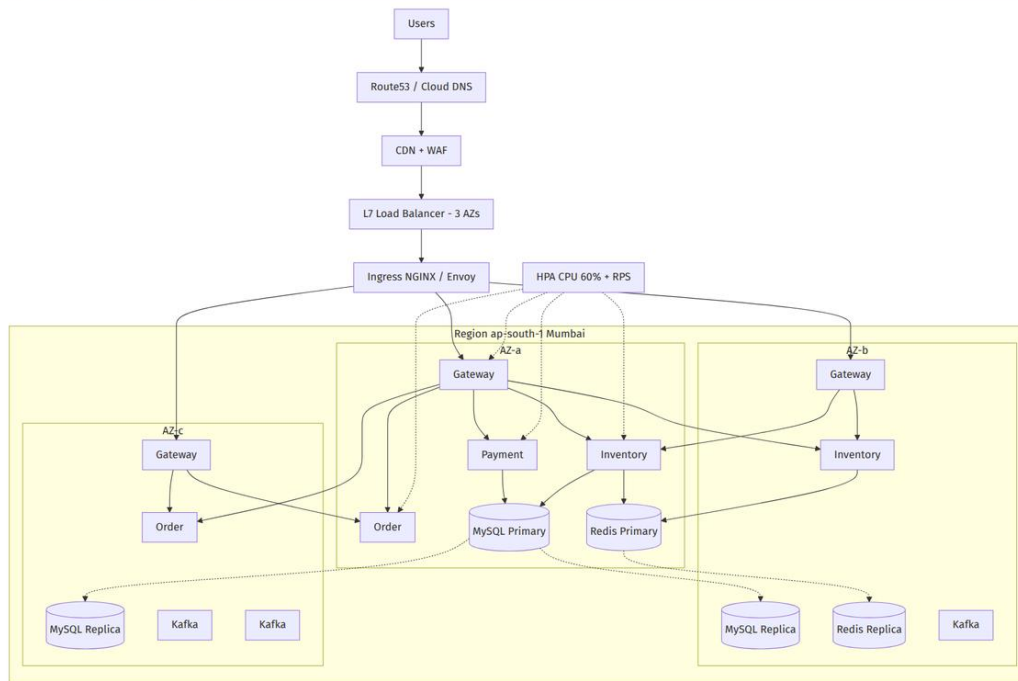


Figure 4 — Deployment Multi-AZ Kubernetes

4. Class Diagram (LLD)

InventoryService depends on StockGate (implemented by RedisLuaStockGate) and Reservation entity with status machine. PaymentService uses PaymentStrategy (Card/UPI) and CircuitBreakerDecorator. OrderService consumes PaymentSucceededEvent and creates Order idempotently.

4. Class Diagram (LLD)

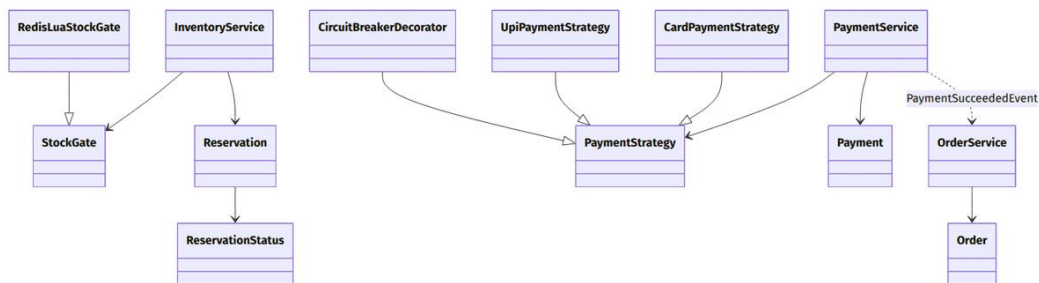


Figure 5 — Class Diagram

5. Sequence Diagrams

5.1 Successful Purchase

Client reserves with JWT + X-Idempotency-Key → Gateway validates → Inventory EVALSHA reserve.lua → MySQL conditional UPDATE → reservationId returned → charge → CAS to PAYMENT_PENDING → gateway success → outbox → Order created.

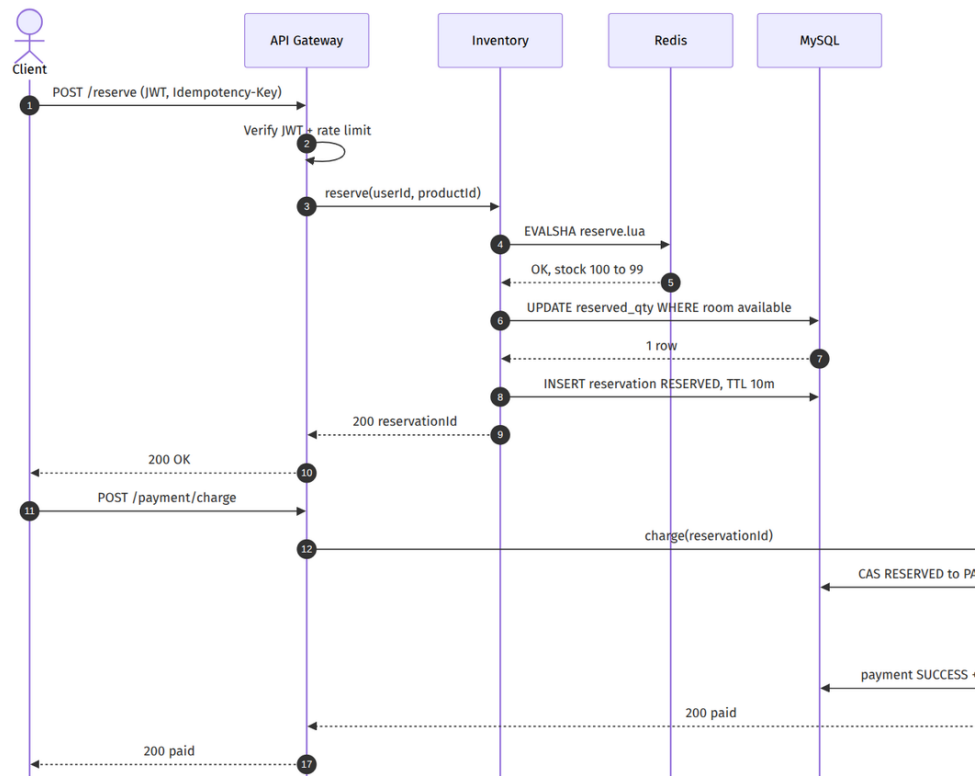


Figure 6 — Successful Purchase

5.2 Out-of-Stock (Requests 101–10,000)

After stock hits zero, a local sold-out cache (250 ms TTL) short-circuits most requests with HTTP 409. Cache misses hit Redis Lua and receive OUT_OF_STOCK. Zero MySQL queries on the reject path.

Out-of-Stock Sequence (Requests 101 to 10000)

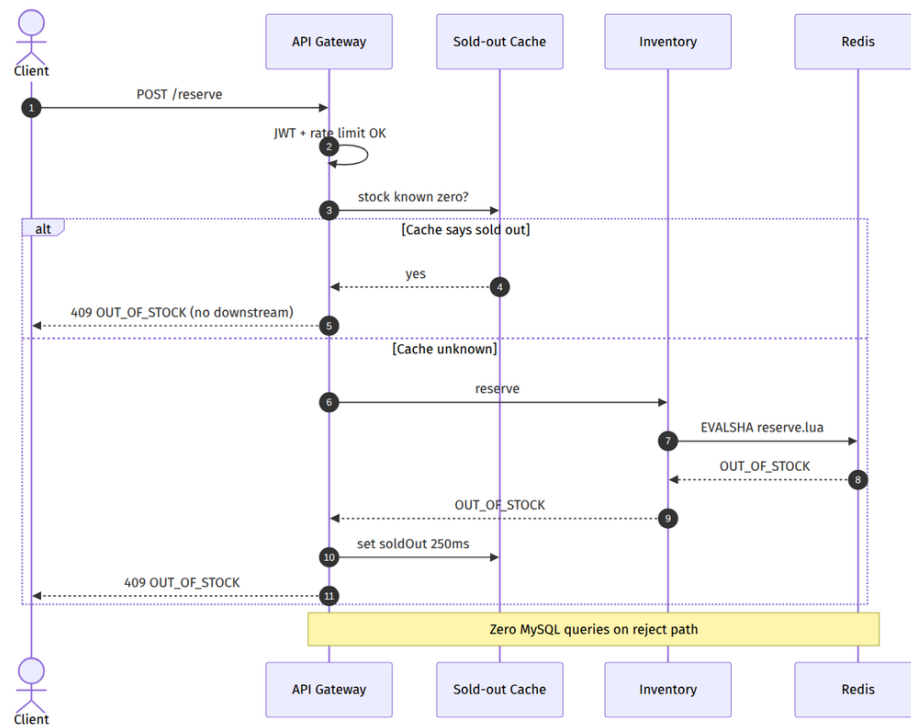


Figure 7 — Out-of-Stock

5.3 Payment Failure & Expiry

Gateway decline: payment FAILED + outbox → Release Worker restores stock via release.lua. Abandoned cart: Expiry Sweeper finds expired RESERVED rows, CAS to EXPIRED, restores stock. Late success after expiry triggers automatic refund — stock is never oversold.

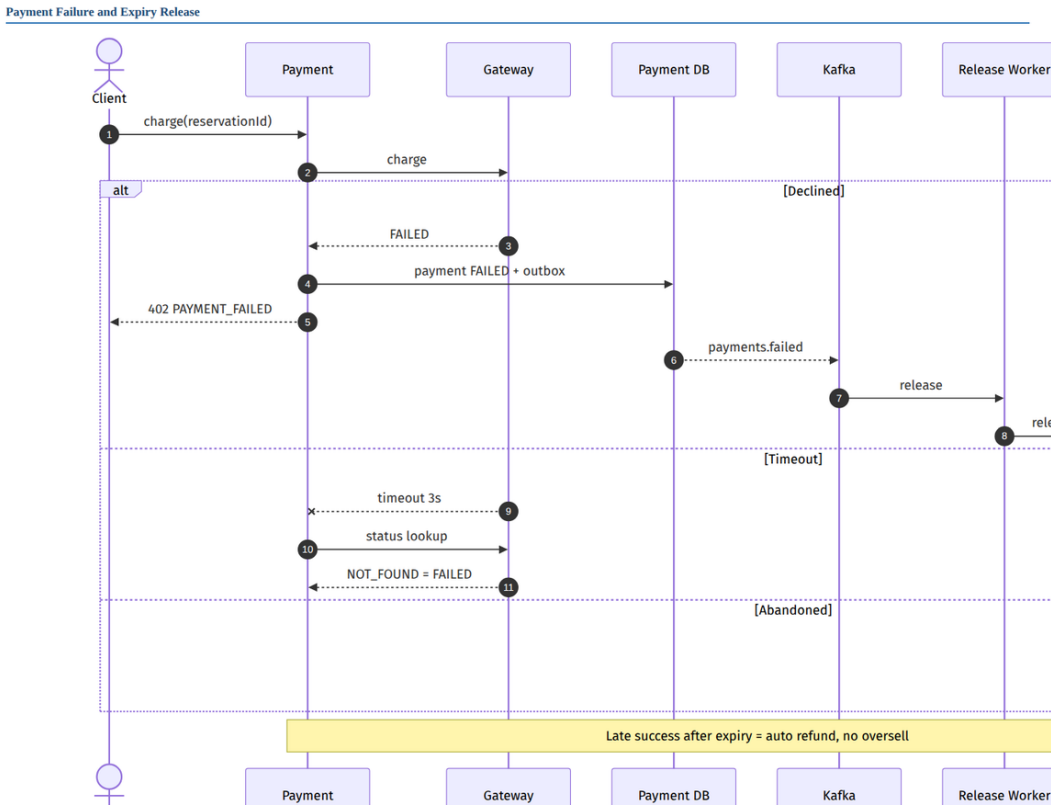


Figure 8 — Payment Failure & Expiry

5.4 Order Outage Recovery

Payment commits Outbox even if Order service is down for 30 seconds. Debezium → Kafka retains the event. When Order recovers, the idempotent consumer creates the order exactly once. Customer polling moves from 202 Accepted to CONFIRMED.

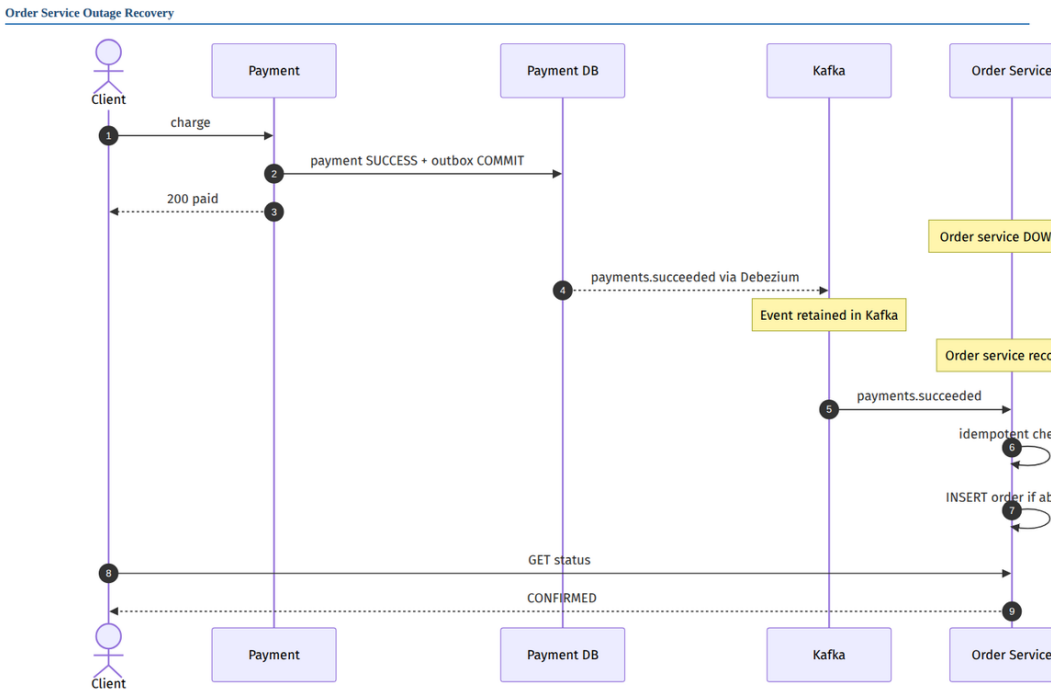


Figure 9 — Order Outage Recovery

6. State Diagrams

6.1 Reservation State Machine

AVAILABLE → RESERVED (Lua + DB) → PAYMENT_PENDING (charge started) → CONFIRMED (payment success). Side paths: EXPIRED or RELEASED return the unit to AVAILABLE.

Inventory Reservation State Machine

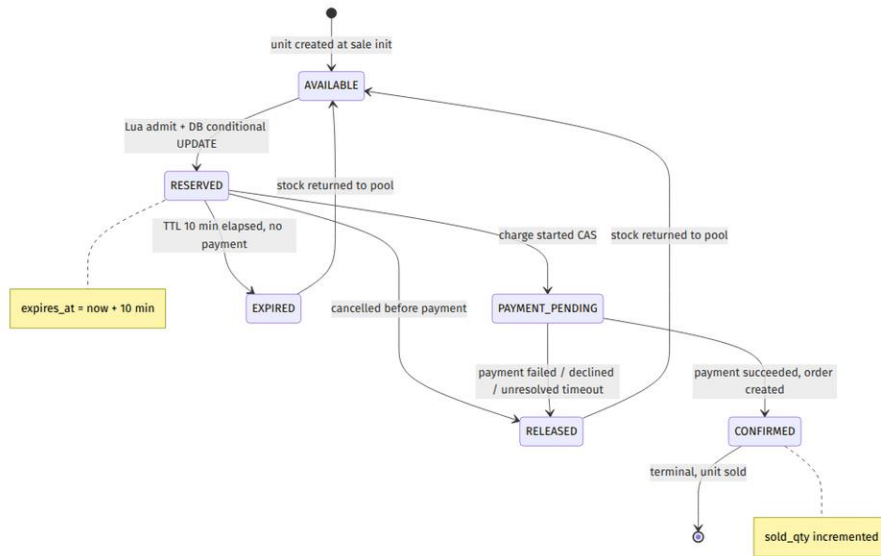


Figure 10 — Reservation States

6.2 Order Lifecycle

CREATED → PROCESSING → SHIPPED → DELIVERED. Cancellation allowed before dispatch.

Order Lifecycle

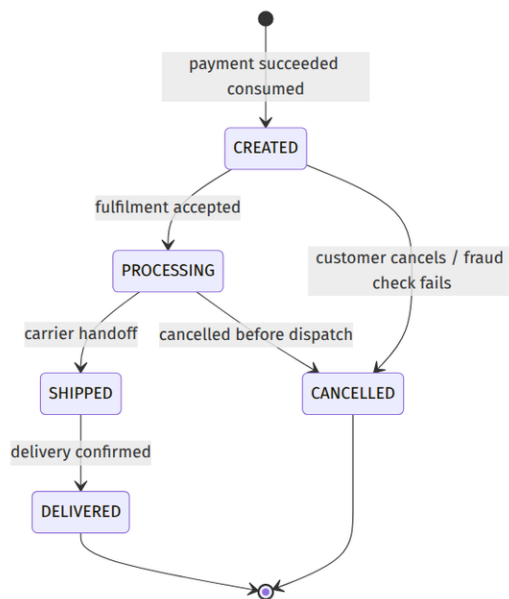


Figure 11 — Order Lifecycle

7. ER Diagram & Database

Core entities: PRODUCT, CUSTOMER, INVENTORY (versioned counters), INVENTORY_RESERVATION (status, expires_at, version, idempotency_key), ORDERS, PAYMENT, IDEMPOTENCY_LOG. Stock invariant enforced by CHECK constraint and conditional UPDATES. PAYMENT links to ORDERS after successful charge.

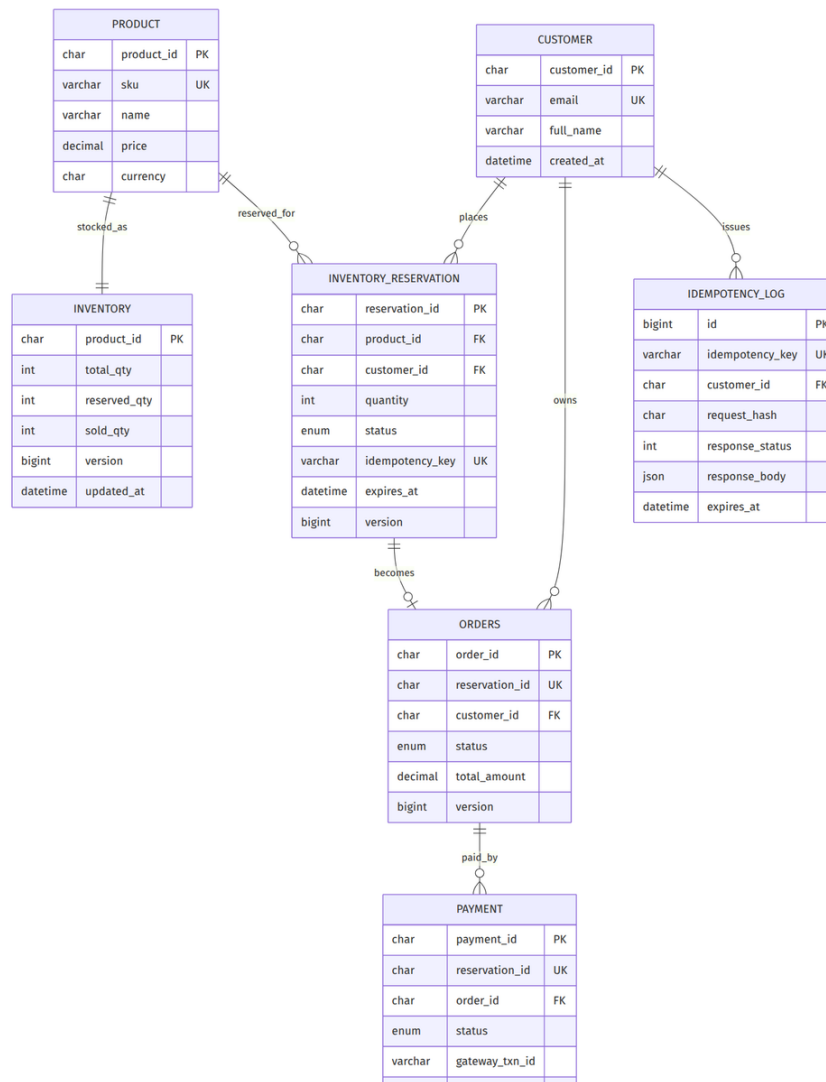


Figure 12 — ER Diagram

Critical SQL — no oversell

UPDATE inventory SET reserved_qty = reserved_qty + 1, version = version + 1 WHERE product_id = ? AND reserved_qty + sold_qty + 1 <= total_qty AND version = ?;

Must return affected_rows = 1. Combined with Redis Lua gate and DB CHECK constraint, this is the three-layer oversell guarantee.

Indexes

idx_reservation_expiry (status, expires_at) for Sweeper; UNIQUE(idempotency_key) on reservation and payment; UNIQUE(reservation_id) on orders and payment.

8. SOLID & Design Patterns

Principle / Pattern	Application in SALESTORM
Single Responsibility	Inventory reserves; Payment charges; Order creates
Open / Closed + Strategy	PaymentStrategy — Card / UPI / NetBanking without changing service
Liskov Substitution	RedisLuaStockGate is a drop-in StockGate
Interface Segregation / DIP	Narrow repositories; services depend on abstractions
State + Compare-and-Set	Reservation status transitions with version column
Circuit Breaker	Decorator around payment gateway calls
Transactional Outbox	Payment success → Order via Kafka with compensation

9. Scalability, Reliability & Security

- Horizontal scale: Gateway, Inventory, Payment, Order pods with HPA (CPU + RPS)
- Redis cluster for hot stock path; MySQL as durable source of truth
- Rate limit + sold-out cache absorb the 10,000-user storm
- Timeouts, retries with jitter, circuit breakers on external gateway
- Idempotency keys + UNIQUE(reservation_id) prevent double side-effects
- JWT auth, WAF, rate limits; no card data stored in platform
- Observability: reservation_success, out_of_stock, payment_failed, kafka_lag; structured logs; tracing

10. Architecture Decision Records

ADR-001 — Redis Lua as primary stock gate

Context: 10,000 concurrent increments against stock of 100. Decision: Single-threaded Lua script on Redis for atomic try-reserve; MySQL conditional UPDATE as durable backup. Consequences: Sub-millisecond rejects for losers; fail-closed if Redis is unavailable (sale pauses, never oversells).

ADR-002 — Transactional Outbox for payment → order

Context: A successful payment must create an order even if the Order service is temporarily down. Decision: Write payment row and outbox event in one database transaction; Debezium streams to Kafka; Order service consumes idempotently. Consequences: At-least-once delivery with exactly-once business effect; no lost paid orders.

11. Jury Defense Cheat-Sheet

- Zero overselling = Redis Lua gate + MySQL conditional UPDATE + CHECK constraint (three layers)
- 10,000 vs 100: only ~100 requests reach MySQL; ~9,700 rejects cost ~0.1 ms Redis each
- Idempotency: X-Idempotency-Key bound to customer + body hash; UNIQUE(reservation_id) on payment and order
- Payment success + Order down 30 s: Outbox + Debezium + Kafka; customer sees 202 then CONFIRMED
- Redis dies: fail closed (503 SALE_PAUSED); rebuild stock from MySQL; oversell still impossible
- Late payment after expiry: automatic refund; stock is never oversold